

Alunos : Lucas do Carmo Maia e Théo Castro Moreira

1) Verdadeiro ou Falso:

A. **(Verdadeiro)** No Bubble Sort, se uma iteração completa não realizar nenhuma troca, o algoritmo pode ser finalizado.

B. **(Falso)** No Selection Sort, o menor elemento não ordenado é escolhido e colocado na posição correta, não o maior.

C. **(Falso)** Tanto o Insertion Sort quanto o Selection Sort podem precisar deslocar elementos, mas de forma diferente.

D. **(Falso)** Quicksort é um método eficiente, não simples.

E. **(Falso)** Métodos como Merge Sort e Quick Sort podem usar estruturas auxiliares (como listas temporárias).

F. **(Verdadeiro)** O Merge Sort divide uma lista em sublistas, ordena-as e as combina recursivamente.

G. **(Falso)** No Quick Sort, após a partição, o pivô fica na posição correta com os menores à esquerda e os maiores à direita.

H. **(Falso)** No Heap Sort, o maior elemento é removido do heap, não o menor.

I. **(Verdadeiro)** No Bubble Sort, o maior elemento é colocado na posição correta após a primeira iteração completa.

J. **(Verdadeiro)** Exemplos de métodos eficientes são Merge Sort, Heap Sort e Quick Sort.

K. **(Verdadeiro)** No Shell Sort, a escolha do gap inicial altera o número de comparações e trocas.

L. **(Verdadeiro)** Durante a fase de divisão no Merge Sort, a lista original é dividida em duas partes de tamanhos iguais ou quase iguais.

M. **(Verdadeiro)** No Quick Sort, a partição em torno de um pivô separa os elementos em duas sublistas: menores e maiores que o pivô.

N. **(Falso)** No Bubble Sort, elementos já ordenados não serão trocados se estiverem na posição correta.

O. **(Falso)** A ordenação externa é usada quando a lista não cabe na memória principal.

P. **(Verdadeiro)** Métodos de ordenação internos podem ser classificados como simples e eficientes, sendo os eficientes melhores para vetores maiores.

2) Ordenação do vetor $V = [8, 7, 6, 1, 2, 4, 3]$:

A. Bubble Sort

Passo a passo do Bubble Sort: 1ª iteração: [7, 6, 1, 2, 4, 3, 8]

2ª iteração: [6, 1, 2, 4, 3, 7, 8]

3ª iteração: [1, 2, 4, 3, 6, 7, 8]

4ª iteração: [1, 2, 3, 4, 6, 7, 8]

$C(n) = 21$

$M(n) = 14$

B. Selection Sort

Passo a passo do Selection Sort: 1ª iteração: [1, 7, 6, 8, 2, 4, 3]

2ª iteração: [1, 2, 6, 8, 7, 4, 3]

3ª iteração: [1, 2, 3, 8, 7, 4, 6]

4ª iteração: [1, 2, 3, 4, 7, 8, 6]

5ª iteração: [1, 2, 3, 4, 6, 8, 7]

6ª iteração: [1, 2, 3, 4, 6, 7, 8]

$C(n) = 21$

$M(n) = 9$

C. Insertion Sort

Passo a passo do Insertion Sort: 1ª iteração: [7, 8, 6, 1, 2, 4, 3]

2ª iteração: [6, 7, 8, 1, 2, 4, 3]

3ª iteração: [1, 6, 7, 8, 2, 4, 3]

4ª iteração: [1, 2, 6, 7, 8, 4, 3]

5ª iteração: [1, 2, 4, 6, 7, 8, 3]

6ª iteração: [1, 2, 3, 4, 6, 7, 8]

$C(n) = 15$

$M(n) = 15$

D. Shell Sort (gap = 5)

Passo a passo do Shell Sort: 1ª iteração (gap 5): [3, 7, 6, 1, 2, 8, 4]

2ª iteração (gap 2): [3, 2, 6, 1, 7, 4, 8]

3ª iteração (gap 1): [1, 2, 3, 4, 6, 7, 8]

$C(n)$ e $M(n)$ dependem dos gaps escolhidos, mas geralmente são melhores que o Insertion Sort.

E. Merge Sort

Passo a passo do Merge Sort:

Divide em [8, 7, 6, 1] e [2, 4, 3]

Divide em [8, 7] e [6, 1] e [2, 4] e [3]

Combina em [7, 8], [1, 6], [2, 4], [3]

Combina em [1, 6, 7, 8] e [2, 3, 4]

Combina em [1, 2, 3, 4, 6, 7, 8]

$C(n) = 18$

$M(n) = 18$

F. Quick Sort (pivô = 1º elemento)

Passo a passo do Quick Sort:

Pivô 8: [7, 6, 1, 2, 4, 3 | 8]

Pivô 7: [6, 1, 2, 4, 3 | 7 | 8]

Pivô 6: [1, 2, 4, 3 | 6 | 7, 8]

Pivô 1: [1 | 2, 4, 3 | 6, 7, 8]

Pivô 2: [1 | 2 | 4, 3 | 6, 7, 8]

Pivô 4: [1 | 2 | 3 | 4 | 6, 7, 8]

C(n) = 15

M(n) = 15

G. Heap Sort

Passo a passo do Heap Sort:

Construir max-heap: [8, 7, 6, 1, 2, 4, 3]

Troca 8 com 3, e reordena heap: [7, 3, 6, 1, 2, 4 | 8]

Troca 7 com 4, e reordena heap: [6, 4, 3, 1, 2 | 7, 8]

Troca 6 com 2, e reordena heap: [4, 2, 3, 1 | 6, 7, 8]

Troca 4 com 1, e reordena heap: [3, 2, 1 | 4, 6, 7, 8]

Troca 3 com 1, e reordena heap: [2, 1 | 3, 4, 6, 7, 8]

Troca 2 com 1: [1 | 2, 3, 4, 6, 7, 8]

C(n) = 15

M(n) = 15